

DESARROLLO BACKEND III (PBY2203)

Informe técnico – Evaluación Final Transversal

Autor: Miguel Castillo

Índice

1. Introducción y contexto....3
1.1 Objetivos....3
1.2 Alcance y supuestos....3
2. Identificación de procesos clave....4
3. Propuesta de arquitectura....5
3.1 Requerimientos clave del negocio....5
3.2 Decisiones tecnológicas y justificación....5
3.3 Diagrama de arquitectura....7
3.4 Diagrama de casos de uso....8
4. Parte 1: Migración de procesos batch....9
4.1 Jobs implementados....9
4.2 Manejo de errores, paralelismo y reejecución....10
4.3 Equivalencia con el sistema legacy y verificación....10
5. Parte 2: Patrón Backend for Frontend....12
5.1 Autenticación, autorización y comunicaciones seguras....12
6. Parte 3: Microservicios resilientes con Spring Cloud....14
6.1 Microservicios de negocio....14
6.2 Spring Cloud: configuración, descubrimiento, balanceo y enrutamiento....14
6.3 Seguridad OAuth2....14
6.4 Resiliencia con Resilience4j....15
6.5 Mensajería asíncrona con Kafka....15
6.6 Consistencia de datos en entorno distribuido....15
6.7 Monitorización y registro....16
7. Docker, escalabilidad horizontal y nube....17
7.1 Contenedores y orquestación....17
7.2 Escalabilidad horizontal....17
7.3 Preparación para AWS....17
8. Documentación entregada....18
9. Comparación con el sistema legacy....19
10. Desafíos, soluciones y mejoras propuestas....20
10.1 Desafíos enfrentados y soluciones....20
10.2 Propuestas de mejora y próximos pasos....20

1. Introducción y contexto

El Banco XYZ opera desde hace más de 30 años un sistema de gestión bancaria legacy construido en COBOL y scripts Shell sobre servidores mainframe. Este sistema concentra operaciones críticas (procesamiento de transacciones, estados de cuenta, cálculo de intereses y gestión de clientes), pero presenta limitaciones de escalabilidad, altos costos de mantenimiento y dificultad para integrar nuevas tecnologías y canales.

Este informe describe la solución desarrollada por Miguel Castillo: una plataforma moderna basada en microservicios con Spring Boot y Spring Cloud, procesos batch migrados a Spring Batch, el patrón Backend for Frontend (BFF) para tres canales (web, móvil y cajeros automáticos), seguridad con OAuth2, resiliencia con Resilience4j, mensajería asíncrona con Kafka y empaquetado en Docker listo para AWS.

1.1 Objetivos

- Reescribir los procesos batch legacy en Spring Batch con manejo de errores, reintentos, paralelismo y reejecución automática.
- Separar el monolito en microservicios independientes (Clientes, Cuentas y Pagos) con configuración centralizada y descubrimiento dinámico.
- Entregar respuestas optimizadas y seguras por canal mediante tres BFF independientes.
- Garantizar seguridad distribuida (OAuth2/JWT, HTTPS), tolerancia a fallos y escalabilidad horizontal.

1.2 Alcance y supuestos

El alcance cubre las tres partes solicitadas en la evaluación. Como datos de entrada se utilizó un conjunto de prueba con el formato descrito en la sección 4 (20.000 transacciones y 200 cuentas); los archivos pueden reemplazarse por los del repositorio fin_legacy_data ajustando el mapeo de columnas. La autenticación implementada es de tipo cliente-aplicación (client_credentials) por canal; la identidad del usuario final queda como mejora (sección 10).

2. Identificación de procesos clave

Se identificaron cinco procesos críticos para la migración desde el sistema legacy hacia una arquitectura de microservicios. Cada uno resuelve un problema concreto del sistema actual y se materializa en componentes específicos de la solución.

#	Proceso clave	Problema en el legacy	Solución implementada	Componentes
1	Migración de procesos batch a Spring Batch	Procesos COBOL/Shell secuenciales, sin reintentos ni paralelismo, difíciles de mantener	3 jobs chunk-oriented con retry, skip, multi-hilo y reejecución automática	batch-service
2	División del monolito en microservicios	Escalado y mantenimiento complejos; un fallo afecta a todo	Servicios de Clientes, Cuentas y Pagos con BD propia, Config Server, Eureka y Gateway	clientes/cuentas/ pagos-service, config-server, eureka-server, api-gateway
3	Patrón Backend for Frontend	Todos los canales reciben los mismos datos; el frontend depende del monolito	Un BFF por canal con payload y reglas propias	bff-web, bff-movil, bff-atm
4	Seguridad distribuida con Spring Cloud Security	Seguridad centralizada y limitada	OAuth2/JWT con scopes y TTL por canal; HTTPS en el punto de entrada; cada servicio valida el token	auth-server, common (resource server), api-gateway
5	Mensajería asíncrona con Kafka	Acoplamiento síncrono entre módulos	Eventos de transacciones completadas y alertas de seguridad en tópicos Kafka	pagos-service (productor), notificaciones-service (consumidor)

3. Propuesta de arquitectura

3.1 Requerimientos clave del negocio

A partir del contexto del Banco XYZ se identificaron cinco requerimientos clave. Cada uno se vincula con decisiones de arquitectura concretas que se justifican en la sección 3.2.

Requerimiento de negocio	Justificación	Decisión de arquitectura
R1. Escalabilidad y disponibilidad	La demanda bancaria varía (cierres de mes, campañas) y el mainframe solo escala verticalmente con alto costo.	Microservicios sin estado, descubrimiento con Eureka, balanceo client-side y Gateway; escalado horizontal con Docker/ECS.
R2. Seguridad y cumplimiento	Se manejan datos financieros y personales; cada canal tiene un nivel de riesgo distinto.	OAuth2/JWT con scopes y TTL por canal, validación en cada servicio, HTTPS, bloqueo de operaciones fuera del canal.
R3. Continuidad operacional (resiliencia)	Un fallo en un módulo no debe detener los pagos ni dejar al cliente sin respuesta.	Resilience4j (Retry, CircuitBreaker, timeouts) con comportamientos alternativos; Saga con compensación.
R4. Eficiencia operativa de procesos masivos	Los procesos nocturnos deben terminar en ventanas acotadas y ser re-ejecutables.	Spring Batch con chunks, paralelismo, UPSERT idempotente y reejecución automática.
R5. Agilidad para nuevos canales	Web, móvil y ATM evolucionan con ritmos distintos y necesitan datos distintos.	Patrón BFF: equipos frontend y backend desarrollan en paralelo con contratos por canal.

3.2 Decisiones tecnológicas y justificación

Decisión	Alternativas	Justificación
Spring Boot 3 / Java 17 + Spring Cloud 2023.0	Quarkus, Micronaut	Ecosistema maduro, integración nativa con Config, Eureka, Gateway, OAuth2 y Resilience4j; estándar en banca.
Spring Cloud Config (perfil native)	Variables de entorno, Consul	Configuración centralizada y versionable; los servicios no contienen URLs ni parámetros de resiliencia.
Eureka + Spring Cloud LoadBalancer	Kubernetes DNS, Consul	Detección dinámica y round-robin

		client-side; permite --scale sin cambiar configuración.
Spring Cloud Gateway con HTTPS	Nginx, AWS API Gateway	Enrutamiento lb:// por canal, un único punto de entrada seguro y portable a AWS ALB.
OAuth2 (Spring Authorization Server) + JWT	Keycloak, Cognito	Control total de scopes y TTL por canal; tokens verificables por cada servicio sin consultar al emisor (JWKS).
Resilience4j	Hystrix (obsoleto), Sentinel	Integración con Spring Boot 3, configuración declarativa, métricas y health indicators.
Kafka (KRaft)	RabbitMQ, JMS	Tópicos persistentes y particionados, consumidores independientes, retención de eventos.
PostgreSQL (una BD por servicio)	BD compartida, H2	Aislamiento de datos entre microservicios, transacciones ACID y bloqueos de fila; equivalente a Amazon RDS.
Spring Batch	Quartz + scripts	Modelo estándar de jobs/steps con repositorio de ejecuciones, reintentos, skip y reinicio.
Docker + docker-compose	Instalación manual	Entorno reproducible y portable a ECS/EKS.

3.3 Diagrama de arquitectura

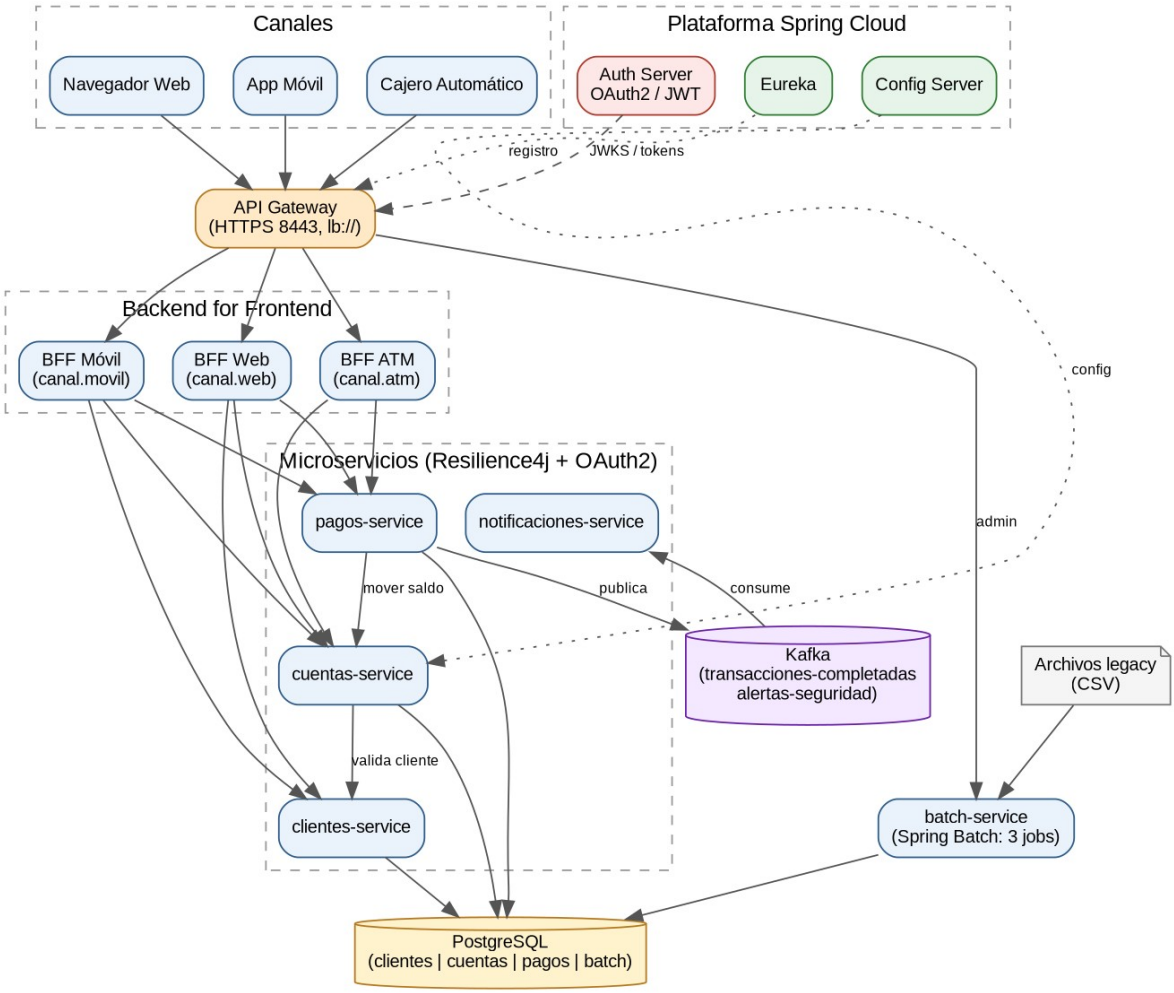


Figura 1. Arquitectura general de la solución

3.4 Diagrama de casos de uso

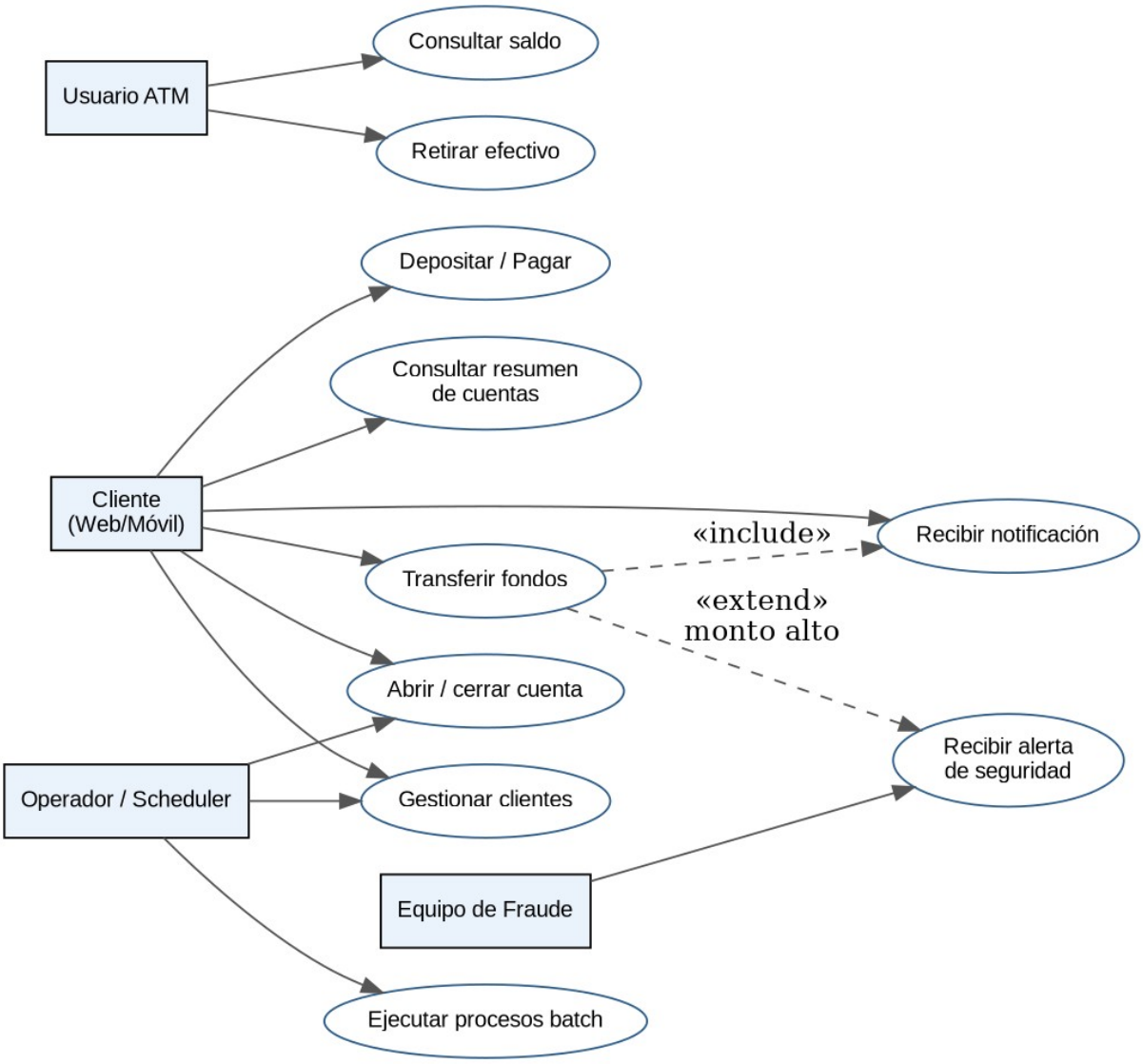


Figura 2. Casos de uso principales por actor

4. Parte 1: Migración de procesos batch

Los tres procesos legacy fueron reescritos como jobs de Spring Batch dentro del módulo batch-service. Los jobs se ejecutan bajo demanda mediante un endpoint protegido (scope banco.admin) y pueden programarse con un scheduler externo (por ejemplo EventBridge en AWS).

4.1 Jobs implementados

Job	Proceso legacy	Lectura	Procesamiento	Escritura
reporteTransaccionesDiarias	Reporte de Transacciones Diarias	CSV de transacciones (reader sincronizado, multi-hilo)	Detecta anomalías: monto no positivo, monto sobre 5.000.000 o tipo desconocido; filtro opcional por fecha	UPSERT en reporte_transacciones + step de resumen por fecha y tipo (resumen_diario)
calculoInteresesMensuales	Cálculo de Intereses Mensuales	Cuentas desde BD (paginado, thread-safe)	interés = saldo × tasa anual / 12 (ahorro 3 %, préstamo 12 %, corriente 0 %), redondeo HALF_UP	UPSERT en intereses_calculados (cuenta, periodo)
estadosCuentaAnuales	Generación de Estados de Cuenta Anuales	Cuentas desde BD (paginado)	Totaliza depósitos, cargos, movimientos válidos e intereses del año	UPSERT en estados_cuenta (cuenta, año)

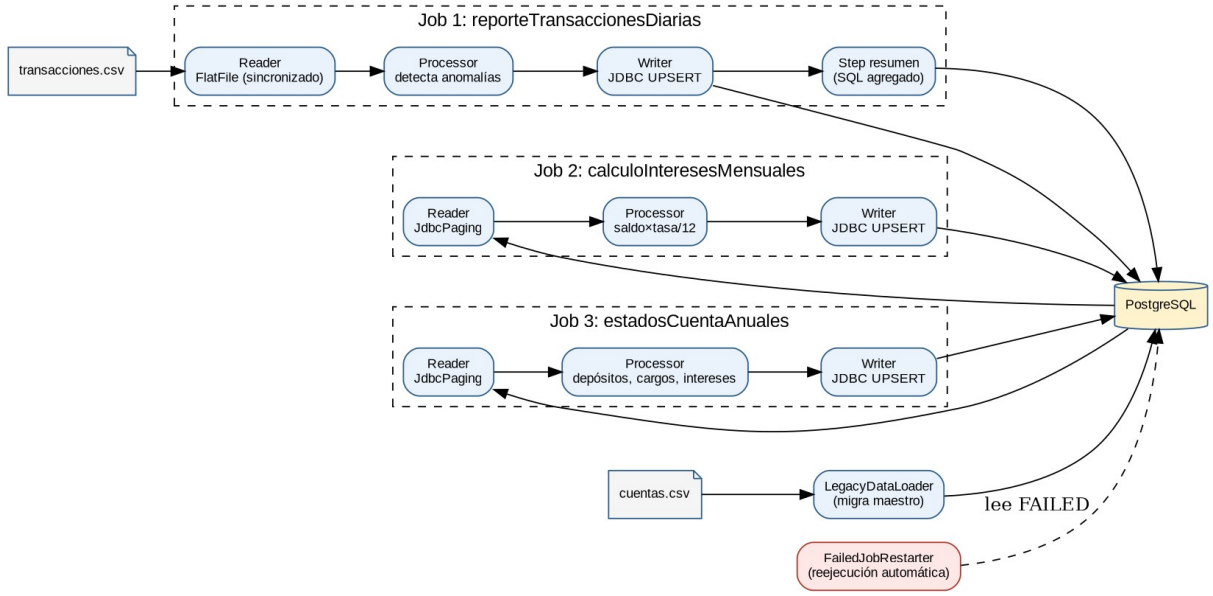


Figura 3. Flujo de los procesos batch

4.2 Manejo de errores, paralelismo y reejecución

- **Reintentos:** fallos transitorios de base de datos (TransientDataAccessException) se reintentan 3 veces con espera fija de 1 segundo.
- **Omisión controlada:** filas corruptas (errores de parseo o fechas inválidas) se omiten hasta un máximo de 50 sin detener el job.
- **Paralelismo y volumen:** steps multi-hilo (4 hilos), chunks de 500 registros y escritura JDBC por lotes; el lector de archivo se sincroniza y el lector de BD es thread-safe.
- **Integridad y consistencia:** toda escritura es un UPSERT (ON CONFLICT DO UPDATE), por lo que reejecutar un job no duplica datos; cada chunk es transaccional.
- **Política de finalización y reejecución automática:** el componente FailedJobRestarter revisa cada 60 s las instancias en estado FAILED y las relanza con los mismos parámetros (hasta 3 reintentos); superado el límite, registra un error para intervención manual. El repositorio de Spring Batch en PostgreSQL evita ejecuciones duplicadas entre instancias.

4.3 Equivalencia con el sistema legacy y verificación

Para asegurar resultados equivalentes se mantuvieron las reglas de negocio del legacy (detección de anomalías, fórmula de interés y totalización anual) y se definieron controles de

verificación: conteo de registros leídos contra escritos más omitidos, y totales por fecha y tipo comparables con el resumen que producía el proceso original.

Con el conjunto de datos de prueba incluido (20.003 líneas de datos), los valores esperados del job reporteTransaccionesDiarias son: 20.000 registros procesados y escritos, 3 filas corruptas omitidas y 39 anomalías detectadas. Estos valores se verifican con el script scripts/generar_evidencia.sh, cuya salida queda registrada en la carpeta evidencias.

Aspecto	Legacy (COBOL/Shell)	Nueva solución (Spring Batch)
Ejecución	Secuencial en mainframe	Multi-hilo, chunks de 500, escalable
Errores	El proceso se detiene o requiere intervención	Retry, skip y reejecución automática
Reproceso	Manual, riesgo de duplicados	Idempotente (UPSERT)
Trazabilidad	Logs de texto	Repositorio de ejecuciones (lecturas, escrituras, omitidos) consultable por API
Operación	Scripts Shell	API protegida y programable en la nube

5. Parte 2: Patrón Backend for Frontend

Se implementaron tres BFF independientes, cada uno como microservicio propio con su contrato, seguridad y reglas de degradación. Los equipos de frontend de cada canal pueden evolucionar su BFF sin afectar a los demás ni al backend de dominio.

Aspecto	BFF Web	BFF Móvil	BFF ATM
Ruta	/web/**	/movil/**	/atm/**
Scope exigido	canal.web	canal.movil	canal.atm
TTL del token	30 min	15 min	5 min
Payload	Completo: cliente, cuentas, movimientos recientes, advertencias	Mínimo: nombre y (número, saldo) por cuenta	Solo saldo y resultado del retiro
Operaciones	Dashboard, transferencias	Resumen, transferencias	Consulta de saldo, retiro
Reglas propias	Agrega datos de 3 servicios	Reduce campos para ahorrar ancho de banda	Máximo 300.000 por retiro, múltiplos de 10.000
Ante fallos	Degradación: responde con advertencias	Degradación: aviso al usuario	Sin degradación: 503 controlado (operación crítica)

5.1 Autenticación, autorización y comunicaciones seguras

- Cada canal tiene su propio cliente OAuth2 con scopes y vigencia distintos; un token de un canal es rechazado (403) en los BFF de otros canales.
- El token se propaga a los microservicios, que validan firma (JWKS del Authorization Server) y el scope banco.read/banco.write.
- El API Gateway expone únicamente HTTPS (puerto 8443) con certificado PKCS12; en producción se reemplaza por un certificado de ACM/CA.
- Sin token la respuesta es 401; los endpoints de actuator expuestos públicamente se limitan a health e info.

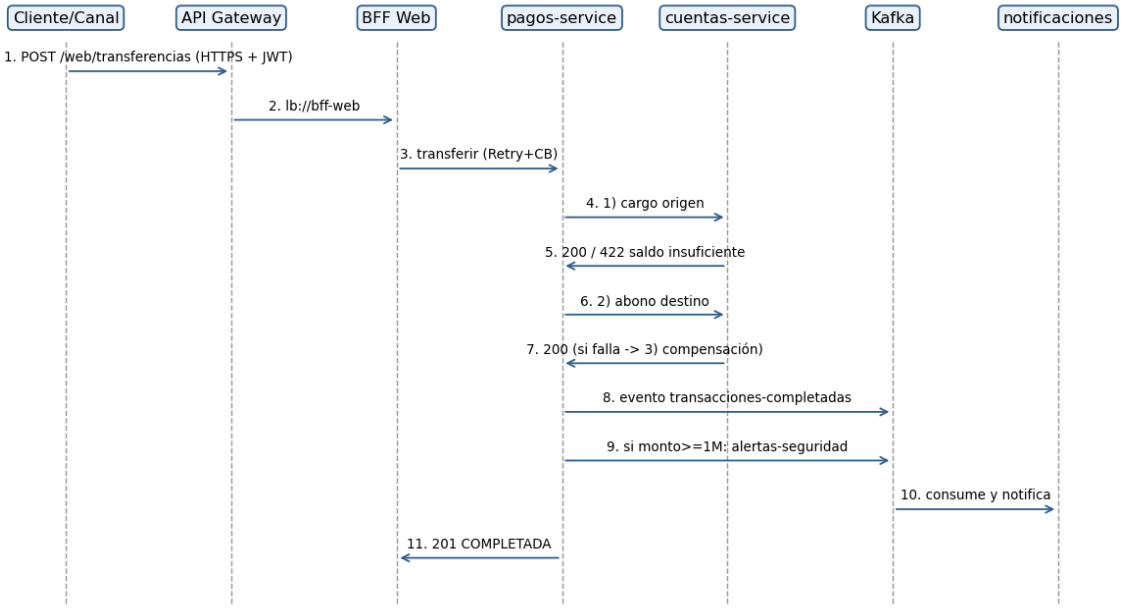


Figura 4. Secuencia de una transferencia desde el canal web

6. Parte 3: Microservicios resilientes con Spring Cloud

6.1 Microservicios de negocio

Servicio	Responsabilidad	Endpoints principales
clientes-service	Información personal y perfiles de clientes	GET/POST /api/clientes, GET/PUT /api/clientes/{id}
cuentas-service	Apertura, cierre y mantenimiento de cuentas	GET /api/cuentas?clienteId=, POST /api/cuentas, POST /{n}/cierre, POST /{n}/movimiento
pagos-service	Pagos, transferencias, depósitos y retiros	POST /api/pagos/transferencia|deposito|retiro|pago, GET /api/pagos?cuenta=
notificaciones-service	Consumo de eventos y envío de notificaciones	Consumidor Kafka; GET /api/notificaciones

6.2 Spring Cloud: configuración, descubrimiento, balanceo y enrutamiento

- **Spring Cloud Config:** toda la configuración compartida (Eureka, JWKS, Kafka, Resilience4j) y específica (puertos, bases de datos, scopes) vive en el Config Server; los servicios solo conocen su nombre y la URL del servidor.
- **Eureka:** los servicios se registran con un identificador único por instancia, de modo que varias réplicas conviven sin conflicto.
- **Balanceo y enrutamiento:** el Gateway enruta por prefijo hacia lb://bff-*, y los clientes HTTP internos usan RestClient con @LoadBalanced (round-robin) sobre nombres lógicos.

6.3 Seguridad OAuth2

El módulo común configura todos los servicios como Resource Server: sesiones sin estado, GET exige el scope de lectura y las demás operaciones el de escritura, configurables por servicio. El batch-service exige banco.admin.

6.4 Resiliencia con Resilience4j

Mecanismo	Configuración	Efecto
CircuitBreaker (clientes, cuentas, pagos)	Ventana 6 llamadas, mínimo 3, umbral 50 %, 15 s abierto, 2 llamadas en half-open	Evita saturar un servicio caído y responde de inmediato
Retry	3 intentos, espera 500 ms	Absorbe fallos transitorios
Timeouts HTTP	Conexión 2 s, lectura 3 s	Evita hilos bloqueados
Excepciones ignoradas	Errores 4xx (HttpClientErrorException)	Errores de negocio (saldo insuficiente, no existe) no abren el circuito
Fallback	ServicioNoDisponibleException → 503	Respuesta controlada; los BFF web/móvil degradan, el ATM falla de forma explícita

6.5 Mensajería asíncrona con Kafka

- **transacciones-completadas:** evento por cada pago, transferencia, depósito o retiro completado; lo consume notificaciones-service.
- **alertas-seguridad:** evento cuando el monto es mayor o igual a 1.000.000 o cuando falla una compensación; se notifica al equipo de fraude.
- **Productor y consumidor:** pagos-service publica de forma asíncrona (no bloquea la transacción); notificaciones-service procesa en tiempo real con su propio grupo de consumo.

6.6 Consistencia de datos en entorno distribuido

- Patrón Saga con compensación: si el abono falla después del cargo, pagos-service revierte el cargo; si la compensación falla, genera una alerta para revisión manual.
- Cada pago queda registrado con estado PENDIENTE, COMPLETADA o FALLIDA, lo que permite auditoría y conciliación.
- Bloqueo pesimista por fila en cuentas-service para evitar saldos inconsistentes cuando varias instancias escriben a la vez; base de datos propia por servicio.

6.7 Monitorización y registro

Todos los servicios exponen actuator (health, métricas, estado de los circuit breakers y reintentos) y registran cada petición con método, ruta, estado, duración e identificador de la instancia que respondió, lo que permite verificar el balanceo y diagnosticar incidentes con docker compose logs o CloudWatch.

7. Docker, escalabilidad horizontal y nube

7.1 Contenedores y orquestación

Un único Dockerfile multi-etapa y parametrizado compila cada módulo con Maven y genera una imagen ligera con JRE 17. El docker-compose.yaml orquesta 13 servicios y la infraestructura (PostgreSQL y Kafka), con healthchecks y dependencias que garantizan el orden de arranque: PostgreSQL y Kafka, Config Server, Eureka, Auth Server, microservicios, BFF y Gateway.

7.2 Escalabilidad horizontal

Los servicios de dominio no publican puertos al host y son stateless, por lo que pueden replicarse con docker compose up --scale cuentas-service=3 --scale pagos-service=2. Eureka registra las nuevas instancias y el balanceador las incorpora automáticamente; los logs muestran qué contenedor atendió cada solicitud.

7.3 Preparación para AWS

Componente local	Servicio AWS
Imágenes Docker	Amazon ECR
docker-compose	Amazon ECS Fargate (desiredCount y Auto Scaling)
PostgreSQL	Amazon RDS (Multi-AZ)
Kafka	Amazon MSK
Gateway HTTPS	Application Load Balancer + ACM
Secretos	AWS Secrets Manager / SSM
Logs y alarmas	CloudWatch
Batch programado	EventBridge Scheduler

El procedimiento detallado está en despliegue.md (opción rápida con EC2 y opción de producción con ECS).

8. Documentación entregada

Entregable	Archivo	Contenido
Código fuente	readme.md y repositorio GitHub	Estructura multi-módulo Maven, descripción de componentes y enlace al repositorio
Informe técnico	Informe_Tecnico.pdf	Este documento
Instrucciones	instrucciones.md	Ejecución y pruebas paso a paso de cada componente
Despliegue en la nube	despliegue.md	Pasos para AWS (EC2 y ECS)
Evidencia	evidencias/evidencia_ejecucion.txt	Salida real de las pruebas, generada con scripts/generar_evidencia.sh
Video	MP4 de 5 a 7 minutos	Presentación con webcam y evidencias

9. Comparación con el sistema legacy

Dimensión	Sistema legacy	Solución nueva
Arquitectura	Monolito en mainframe	Microservicios independientes con BD propia
Escalabilidad	Vertical, costosa	Horizontal (réplicas en Docker/ECS)
Tolerancia a fallos	Un fallo puede afectar a todo	Circuit breaker, retry, fallback y degradación por canal
Canales	Un backend para todos	BFF por canal con payload y seguridad propios
Seguridad	Centralizada y limitada	OAuth2/JWT por canal, HTTPS, validación en cada servicio
Procesos batch	COBOL/Shell secuenciales	Spring Batch paralelo, con reintentos y reejecución automática
Integración	Acoplamiento síncrono	Eventos Kafka
Configuración	Dispersa en scripts	Centralizada en Config Server
Despliegue	Manual en servidores	Contenedores reproducibles y portables a la nube

10. Desafíos, soluciones y mejoras propuestas

10.1 Desafíos enfrentados y soluciones

Desafío	Solución
Consistencia entre cuentas y pagos sin transacciones distribuidas	Saga con compensación, estados de pago y bloqueo pesimista por fila
Fallos en cascada entre servicios	Resilience4j con timeouts, retry, circuit breaker y fallbacks distintos por canal
Distinguir errores de negocio de fallos técnicos	Ignorar los 4xx en el circuit breaker y propagarlos tal cual al cliente
Procesar grandes volúmenes sin perder repetibilidad	Chunks, multi-hilo y escrituras UPSERT idempotentes
Orden de arranque de 13 servicios	Healthchecks, depends_on con condiciones y reinicio ante fallo
Escalar réplicas sin conflicto de puertos	Sin puertos publicados, descubrimiento con Eureka y acceso por Gateway
Seguridad diferenciada por canal	Un cliente OAuth2 por canal con scopes y TTL propios

10.2 Propuestas de mejora y próximos pasos

- Clave de idempotencia (Idempotency-Key) en pagos y patrón Outbox para publicar eventos de forma atómica con la base de datos.
- Autenticación de usuarios finales con Authorization Code + PKCE y MFA; claves de firma persistentes y clientes en base de datos o Cognito.
- Trazabilidad distribuida (OpenTelemetry) y panel de métricas (Prometheus/Grafana).
- Particionado de steps batch (partitioning remoto) y programación con EventBridge; generación de estados de cuenta en PDF.
- Migración a ECS/EKS con Auto Scaling, RDS Multi-AZ, MSK y cifrado de configuración con Secrets Manager.
- Pruebas automatizadas de contrato por BFF y pruebas de carga.